펫 클리닉 3계층 아키텍처 인프라 구축

NAT 게이트웨이 없는 AWS 3계층 구성과 절차서 기반 구축

Cloud 9: 김태준(팀장), 김건형, 김승환, 양진현, 우수빈

AI 서비스 운영을 위한 클라우드 데이터 엔지니어 과정 중간 프로젝트

프로젝트 기간: 2026. 9. 21. ~ 2026. 10. 14. (13일)

2026. 10. 14. (수)

© 2026 Cloud 9

목차

1. 서비스 시나리오, 아키텍처, 팀과 작업 방식, 비용 결정 (김승환)
2. 작업 기반: VPC, 서브넷, 보안 그룹, 엔드포인트 (김태준)
3. WAS: 호환성과 취약점 보완, Arm 기반 배포와 설정 외부화, 가용성과 자동 확장 (김건형)
4. DB 서버: RDS MySQL과 RDS Proxy, DB 보안 구성과 데이터 적재 (우수빈)
5. 웹 서버와 CloudFront (양진현)
6. 부하 시험, 최종 결과물, 마무리 (김승환)
7. 시연 (김건형)

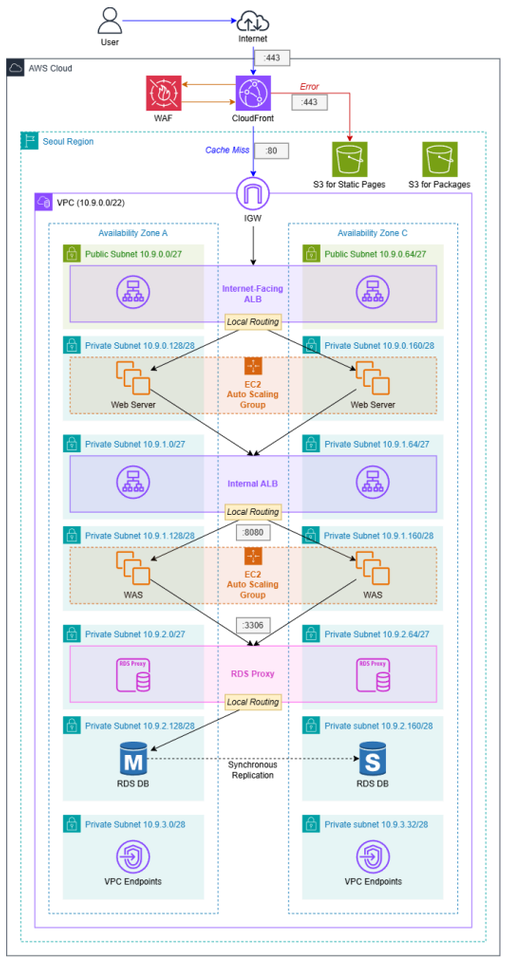
© 2026 Cloud 9

프로젝트 개요와 서비스 시나리오

동물병원 300개 지점을 24시간 운영하는 펫 클리닉이, 병원 직원이 쓸 홈페이지(PetClinic)의 인프라를 AWS에 구축해 달라고 요청했다.

요구 사항	설계에 반영한 것
서버 하나가 고장 나도 2분 안에 정상 서비스	AZ 두 곳에 분산, ALB 상태 확인으로 고장 서버 제외, Auto Scaling이 교체
AZ에 문제가 있으면	임시로 다른 AZ 하나만 사용 (영역 이동)
앱 업데이트 중에도 서비스 중단 없음	새 AMI로 인스턴스 새로 고침, 새 인스턴스를 먼저 띄운 뒤 교체
동시 사용자 600명	인스턴스 사양 결정, k6 부하 시험으로 확인
고객 정보 저장 시 암호화, 백업	RDS 저장 암호화, DB 구간 TLS, 자동 백업 7일
병원 양식의 오류 페이지	CloudFront 사용자 지정 오류 응답과 S3
Apache, Tomcat, OpenJDK 기반	Apache 2.4, Tomcat 9, Amazon Corretto 8 (OpenJDK 배포판)
사용자 규모 확대 (미확정)	WAS 2~4대 자동 조정, AZ B와 D의 주소 대역 예약

아키텍처



사용자 → CloudFront (+ WAF) → 외부 ALB → Apache → 내부 ALB → Tomcat → RDS Proxy → RDS MySQL

계층	구성
웹 서버	Apache, `t4g.micro`, Auto Scaling 그룹
WAS	Tomcat 9 + Java 8, `t4g.small`, 2~4대 자동 조정
DB 서버	MySQL 8.4 Multi-AZ, `db.t4g.small`, RDS Proxy

핵심 결정

- NAT 게이트웨이 없음: AWS 서비스는 VPC 엔드포인트로, 서버 접속은 Session Manager로
- 서버는 모두 프라이빗 서브넷. 퍼블릭 서브넷에는 외부 ALB만
- 전 서버 Arm (Graviton) 인스턴스

그림은 보기 쉽게 EC2를 서브넷 안에 그렸다. 서브넷, 보안 그룹, IP 주소가 붙는 대상은 EC2의 네트워크 인터페이스다.

팀 구성과 작업 방식

이름	주 담당	부 담당
김태준 (팀장)	작업 기반	PM
김건형	WAS	DB 서버
김승환	PM	웹 서버
양진현	웹 서버	작업 기반
우수빈	DB 서버	WAS

발표는 주 담당 영역을 맡는다. 부 담당 영역은 **교차 수행**으로 검토해 서로의 기술 이해도를 맞췄다.

DEV에서 검증하고 PROD에 적용

- **DEV 구축**: 공용(`pet-shared-`) 자원과 개발용 자원으로 먼저 만든다
- **테스트와 검증**: 동작과 근거를 확인한 결정만 남긴다
- **PROD 적용**: 운영용(`pet-prod-`) 자원을 절차서대로 만든다

절차서 10편 + 부록 2편

목적 → 선행 조건 → 절차 → 유의 사항. 모든 단계가 **기대 결과**로 끝나서 누가 따라 해도 같은 결과가 나오는지 확인할 수 있다.

일정과 도구

날짜	한 일
9. 21.	킥오프, 기존 애플리케이션 분석
9. 22. ~ 9. 30.	DEV 구축과 테스트 (추석 연휴 제외)
10. 1. ~ 10. 8.	PROD 구축, 테스트 설계와 검증
10. 9. ~ 10. 11.	교차 수행과 절차서 보완 (연휴)
10. 12. ~ 10. 14.	발표 자료 준비, 최종 발표

용도	도구
인프라 관리	AWS 콘솔, CloudShell, Systems Manager
개발과 실행	IntelliJ IDEA, JDK, Tomcat, MySQL
협업	Jira, Notion, GitHub
시험과 문서화	k6, Quarto

비용 추산과 결정

NAT 게이트웨이를 빼고 엔드포인트로

출구 방식	월 고정 요금 (USD)
NAT 게이트웨이 1개	46.72
AZ마다 NAT 게이트웨이 (권장 구성)	93.44
인터페이스 엔드포인트 6개 (선택)	56.94

- NAT 1개보다는 비싸지만, NAT 1개는 그 AZ가 멈추면 다른 AZ도 막힌다
- 엔드포인트만 두면 **인터넷으로 나가는 길 자체가 없다**

인스턴스 유형: Graviton (t4g)

유형	시간당 (USD)
t3.micro / t4g.micro	0.013 / 0.0104
t3.small / t4g.small	0.026 / 0.0208

- 같은 크기에서 20% 저렴
- 웹은 정적 응답과 전달만 해서 micro, WAS는 JVM 메모리 때문에 small

서울 리전, 730시간 기준. 초안 — 담당자의 추산표로 맞추기

작업 기반

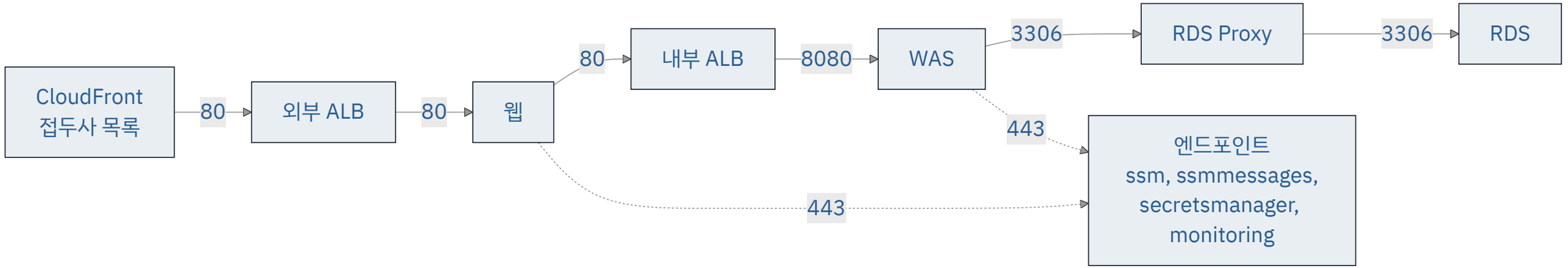
VPC와 서브넷 설계

VPC `10.9.0.0/22`, 서브넷 14개

용도	AZ A	AZ C	크기 이유
외부 ALB	.0.0/27	.0.64/27	ALB는 /27 이상
웹	.0.128/28	.0.160/28	인스턴스만
내부 ALB	.1.0/27	.1.64/27	ALB는 /27 이상
WAS	.1.128/28	.1.160/28	인스턴스만
RDS Proxy	.2.0/27	.2.64/27	여유 IP 10개 필요
DB	.2.128/28	.2.160/28	
엔드포인트	.3.0/28	.3.32/28	

- AZ B, D 자리를 **비워 두어** 나중에 AZ를 늘릴 수 있다
- 라우팅 테이블은 3개: 퍼블릭(인터넷 게이트웨이), 프라이빗, 엔드포인트
- 서브넷마다 AWS가 주소 5개를 예약 → /28은 11개, /27은 27개만 쓸 수 있다
- Docker, EKS 기본 대역과 가정용 공유기 대역을 피했다

보안 그룹 사슬과 관리 접속



- 각 계층은 **바로 앞 계층의 보안 그룹**만 출처로 허용한다
- SSH 포트를 열지 않고 Session Manager로 접속. SSM Agent가 엔드포인트로 먼저 연결한다
- 패키지는 S3 게이트웨이 엔드포인트로 받는다 (Amazon Linux 저장소, 팀 패키지 버킷)

배운 점 (초안 — 본인 말로 바꾸기)

NAT 게이트웨이를 빼면 서버가 부르는 AWS 서비스마다 엔드포인트와 정책이 필요하다. 하나라도 빠지면 그 기능만 조용히 멈춘다.

WAS

호환성 검토와 프레임워크 취약점 보완

버전 호환성

- 기존 Java 8, javax 기반 소스와의 호환성을 고려해 Tomcat 9.0.122, Spring Framework 5.3.39 적용
- Spring BOM으로 Spring 모듈 버전 통일
- Java 패키지 버전 고정으로 검증되지 않은 업데이트 방지

취약점 보완

- 업데이트 후에도 남아 있는 프레임워크 취약점의 실제 사용 조건 확인
- CVE-2024-38820과 관련된 DataBinder 차단 목록 설정을 **입력 허용 목록** 방식으로 변경
- 추가로 확인한 hidden ID 변조 문제에 URL의 `petId` 사용, **보호자와 반려동물 관계 검증** 적용
- 수정 후 전체 테스트 통과, WAR 빌드 확인

Arm 기반 배포와 설정 외부화

Arm 기반 배포

- Amazon Linux 2023, Graviton 기반 `t4g.small` 적용
- **SSM**을 통한 WAR 전송과 Tomcat 배포
- **systemd**를 통한 부팅 시 자동 시작과 서비스 관리
- Arm 환경에서 Java, Tomcat 기동과 PetClinic, DB 연동 확인

설정 외부화

- Tomcat의 `setenv.sh`에서 Parameter Store의 DB 접속 주소와 Secret ARN 조회
- 조회한 설정을 Java 시스템 속성으로 PetClinic에 전달
- Secrets Manager JDBC 드라이버를 통한 DB 자격 증명 조회
- DB 접속 설정 변경 시 WAR 재빌드 없이 Tomcat 재시작으로 반영

가용성 확보와 자동 확장

가용성과 자동 확장

- 2a, 2c 가용 영역에 WAS **분산 배치**
- 내부 ALB를 통한 요청 분산과 상태 확인
- AMI와 시작 템플릿 기반 ASG, CPU 기준 자동 확장 구성

모니터링과 알림

- EC2 CPU와 ALB 응답 시간, 요청량, 오류, 정상 대상 수 확인
- CloudWatch 에이전트로 메모리 사용률 수집
- `HealthyHostCount`가 2 미만이면 CloudWatch 경보와 SNS 이메일 알림
- `whoami.jsp`에서 AWS API로 조회한 **대상 그룹 상태와 현재 요청 경로** 시각화

DB 서버

RDS MySQL과 RDS Proxy

안정적인 DB 운영을 위해 **가용성과 연결 안정성**에 중점을 두었다.

RDS for MySQL 8.4

- **Multi-AZ**: AZ 2a Primary / AZ 2c Standby (동기 복제)
- **MySQL 8.4 LTS**: 장기 지원과 유지보수 고려
- **인스턴스**: `db.t4g.small` / gp3 20 GiB
  - 사전 부하 테스트 연결 사용률: micro 93%, small 56%
- **데이터 보호**: 자동 백업 7일 / KMS 스토리지 암호화

RDS Proxy

- **연결 경로**: WAS → RDS Proxy → RDS
- **연결 풀링과 재사용**: WAS 확장에 따른 DB 연결 증가 대응
- **장애 대응**: Multi-AZ 장애 조치 시 DB 연결 복구 지원
- **TLS 필수**: DB 인증 정보와 전송 데이터 보호
- **최대 연결 비율 80%**: 다른 DB 접속을 위한 연결 여유 확보

DB 보안 구성과 데이터 적재

계정

계정	용도	자격 증명 관리
마스터 `cloud9`	DB 관리	RDS 관리형 Secret
앱 `cloud9-pet`	PetClinic	Secrets Manager

- **최소 권한**: 앱 계정에 `petclinic` DB의 필요한 권한만 부여
- **인증 정보 관리**: Secrets Manager의 Secret 조회
- **접근 제어**: DB 보안 그룹은 RDS Proxy 보안 그룹의 접근만 허용

적재

- **자동 초기화 방지**: WAR 시작 시 SQL 자동 실행 비활성화
- **별도 적재**: 스키마와 초기 데이터 별도 적재
- **테스트 데이터 10만 건**: owners 2만 / pets 3만 / visits 5만

배운 점

RDS Proxy와 Secrets Manager를 연동하는 과정에서 IAM 권한과 보안 그룹 설정이 DB 연결에 미치는 영향을 직접 확인하며, 보안과 접근 제어의 중요성을 배웠다.

웹 서버와 CloudFront

Apache 리버스 프록시

```
<VirtualHost *:80>
    ProxyAddHeaders On
    ProxyPreserveHost On
    RewriteEngine On
    # /petclinic 으로 오면 /petclinic/ 으로 301
    RewriteRule ^/petclinic$ https://%{HTTP_HOST}/petclinic/ [R=301,L]
    # ttl=55: ALB 유휴 제한(60초)보다 먼저 닫는다
    # addressttl=30: ALB IP가 바뀌어도 30초 안에 다시 조회
    ProxyPass        "/petclinic/" "http://<INTERNAL_ALB_DNS>/petclinic/" ttl=55 addressttl=30
    ProxyPassReverse "/petclinic/" "http://<INTERNAL_ALB_DNS>/petclinic/"
</VirtualHost>
```

- `/petclinic/`만 WAS로 넘기고, 나머지는 Apache가 직접 응답
- 내부 ALB는 IP가 아니라 **DNS 이름**으로 가리킨다
- SELinux에서 `httpd_can_network_connect` 허용

CloudFront와 외부 ALB 보호

CloudFront (정액 요금제 Free)

- 사용자 구간만 HTTPS. ALB에는 인증서가 없어 오리진은 HTTP
- 동적 페이지라 캐싱 끔, 쿼리 문자열과 쿠키는 모두 전달
- 오류 시 S3의 공통 오류 페이지 (`/pet_*`, OAC)
- WAF 속도 제한: IP마다 5분에 300건 (차단 모드인지 확인)

외부 ALB에 CloudFront만 닿게

1. 보안 그룹: CloudFront 관리형 **접두사 목록**만 허용
2. 리스너: **비밀 헤더**가 있는 요청만 전달, 나머지는 403

왜 둘 다?

접두사 목록은 **다른 사람의 CloudFront 배포**도 통과시킨다. 비밀 헤더가 우리 배포만 골라낸다.

부하 시험과 마무리

부하 시험 (k6)

- 인터넷이 없는 Arm 인스턴스라 k6 Arm 실행 파일을 **패키지 버킷으로 반입**
- 테스트 데이터 10만 건을 넣고 시험
- 시나리오: 목록 조회, 상세 조회, 등록 (초안)

결과 (초안 — 실제 값으로 채우기)

항목	값
가상 사용자 수	—
초당 요청	—
p95 응답 시간	—
오류율	—
WAS 확장 여부	—

배운 점과 남은 과제

배운 점 (초안)

- 구축보다 **이유를 설명하는 것**이 어렵다. 모든 설정 값에 근거를 남겼다
- 절차서를 남이 따라 해 보면 빠진 단계가 드러난다

남은 과제

- ssm 엔드포인트가 AZ A에만 있어, AZ A 장애 때 새 WAS가 설정을 못 읽을 수 있다 → AZ C 추가
- WAF 속도 제한을 모니터 모드에서 **차단 모드로** 전환
- WAS 정상 대상 수 경보 (SNS 이메일)
- 수작업 구축을 IaC로 옮기기

최종 결과물

- **설계도**: 3계층, AZ 두 곳, NAT 게이트웨이 없는 구성
- **IP 주소 지도와 보안 그룹**: 서브넷 14개, 앞 계층만 허용하는 보안 그룹 사슬
- **VPC 엔드포인트**: ssm, ssmmessages, secretsmanager, monitoring, S3
- **절차서**: 구축 단계 전체와 정책 파일, 설정 파일 (GitHub 저장소와 Wiki)
- **WAS 대시보드**: 요청을 처리한 WAS와 대상 상태 → 시연에서 확인

시연

시연 영상

시연 영상 자리. PDF에는 시연 화면 캡처를 넣는다.

감사합니다.

Cloud 9: 김태준(팀장), 김건형, 김승환, 양진현, 우수빈

부록

보충 설명용 슬라이드

월 고정 요금 추산

항목	이 설계의 수량	월 금액 (USD)
RDS DB 인스턴스 (MySQL, Multi-AZ)	`db.t4g.small` 1대	74.46
인터페이스 VPC 엔드포인트	엔드포인트와 AZ 조합 6개	56.94
EC2 인스턴스	웹 micro 2대, WAS small 2대	45.55
ALB	2개, 각 1 LCU로 가정	44.53
RDS Proxy	DB 인스턴스 vCPU 2개	26.28
외부 ALB 공인 IPv4 주소	2개로 추정	7.30
RDS 스토리지 (gp3, Multi-AZ)	20 GB	5.24
EBS 루트 볼륨 (gp3)	8 GB × 4	2.92
Secrets Manager	비밀 2개	0.80
합계		**약 264.02**

서울 리전, 730시간, 세금 제외. 사용량 요금(데이터 전송, 로그)과 CloudFront는 빠져 있다.

타임아웃 사슬

연결 구간	보내는 쪽 (기본값)	받는 쪽 (기본값)	맞추는 방향
CloudFront → 외부 ALB	keep-alive 5초	ALB 유휴 60초	기본값 그대로 안전
외부 ALB → Apache	ALB 유휴 60초	`KeepAliveTimeout` 5초	받는 쪽을 60초보다 길게
Apache → 내부 ALB	mod_proxy `ttl`	ALB 유휴 60초	`ttl`을 60초보다 짧게 (`ttl=55`)
내부 ALB → Tomcat	ALB 유휴 60초	`keepAliveTimeout` 20초	받는 쪽을 60초보다 길게
Tomcat → RDS Proxy	커넥션 풀 유휴 연결	유휴 클라이언트 연결 1,800초	풀을 1,800초보다 짧게

규칙

한 연결의 두 끝에서 받는 쪽의 유휴 시간을 보내는 쪽보다 길게 둔다. 받는 쪽이 먼저 닫으면 이미 닫힌 연결에 요청이 실려 ALB가 502를 낸다.

응답 대기 상한은 CloudFront 30초, ALB 60초다. 그래서 40초 걸리는 조회는 CloudFront가 먼저 끊는다. GET과 HEAD는 다시 시도하고, POST는 다시 시도하지 않는다.

장애 시 복구 시간

구성	복구 시간 (RTO)에 관련된 사실	데이터 손실 (RPO)에 관련된 사실
Multi-AZ DB 인스턴스 (현재)	장애 조치 보통 60~120초. RDS Proxy가 최대 66% 줄인다	다른 AZ의 대기 인스턴스로 동기 복제
Multi-AZ DB 클러스터	보통 35초 미만	읽기 인스턴스 하나 이상이 확인해야 커밋
Aurora	복제본 승격 보통 60초 미만	저장 노드 6개에 AZ를 넘어 동기 복제
자동 백업에서 특정 시점 복구	문서에 수치 없음	트랜잭션 로그를 5분마다 S3에 올림. 보존 0~35일

측정한 값

- 장애 조치 시험의 SQL 연결 복구: RDS Proxy 경유 13초, 직접 연결 135초
- 전체 웹 서비스 복구 시간이 아니라 SQL 연결 기준

WAS 서버 하나가 고장 나면

- 상태 확인 20초 간격, 2회 실패로 약 40초 뒤 대상에서 빠진다
- 남은 WAS가 요청을 받고, Auto Scaling 그룹이 교체한다

CloudFront 정액 요금제 비교

항목	Free	Pro	Business	Premium
월 요금 (USD)	0	15	200	1,000부터
월 요청 허용량	100만 건	1,000만 건	1억 2,500만 건	5억 건
월 전송 허용량	100 GB	50 TB	50 TB	50 TB
VPC 안의 프라이빗 오리진	없음	없음	있음	있음
WAF 규칙 수	5	25	50	75
IP 기반 속도 제한	있음	있음	있음	있음
봇 관리	없음	없음	있음	있음
가용성 SLA	없음	없음	있음	있음

Free를 골랐다. VPC 오리진은 Business 이상이 필요해서, internet-facing ALB에 CloudFront 접두사 목록과 비밀 헤더 두 겹으로 막았다.